

Università degli studi di Brescia

Dipartimento di Economia e Management

Corso di Laurea Magistrale

Analytics and Data Science for Economics and Management



**UNIVERSITY
OF BRESCIA**

ARTIFICIAL INTELLIGENCE

**Emotion detection from facial expression with
Convolutional Neural Networks**

Professor

Students

Prof.ssa Angela Locoro

**Simone Bosio, Matr. 732753
Nicolò Braghini, Matr. 727460
Massimo Pierini, Matr. 750366**

February 20, 2026

ANNO ACCADEMICO 2025/2026

Contents

Introduction	1
1 Introduction	1
1.1 State of art	1
1.2 Motivation and goals	2
2 Data	3
2.1 Preprocessing and description	3
2.2 Potential improvements	3
3 Methodology	6
3.1 General architecture	6
3.1.1 Convolutional Layer	6
3.1.2 Non-Linearity (Activation Function)	7
3.1.3 Pooling Layer	7
3.1.4 Flattening	7
3.1.5 Fully Connected Layers (Dense Layers)	8
3.1.6 Output Layer (Softmax)	8
3.2 Model details	8
3.2.1 Preprocessing and Augmentation Stage	8
3.2.2 Feature Extraction	10
3.2.3 The Classification Head	11
3.2.4 Training	11
3.3 Proof of concept	12
3.3.1 Confusion Matrix	12
3.3.2 Test with Images	12

3.3.3	Test with Real Images	12
3.3.4	Working Demo	13
4	XAI analysis	19
4.1	Explainability	19
4.2	Regulations and ethics	20
4.3	SHAP analysis	21
5	Conclusions	23
6	Technical appendix	24
	List of Figures	26

1. Introduction

The aim of this project is to develop a deep learning model that can accurately classify facial expressions into one of seven emotional categories: Angry, Disgust, Fear, Happy, Sad, Surprise, and Neutral. The project specifically focuses on implementing and evaluating Convolutional Neural Networks (CNN) for this task, excluding alternative model architectures or additional datasets. Our approach involves training a CNN model from scratch on the FER-2013 dataset (Sambare 2025) to enable the model to learn and distinguish between the specified facial expression categories.

1.1 State of art

CNN-based facial emotion recognition systems employ diverse architectural approaches ranging from custom designs like the two-part FERC framework (Mehendale 2020) and lightweight EmotionNet-X (S. M. A. Abbas, Q. Abbas, and Naqi 2025) to various established architectures including VGG16, ResNet50, and Inception (Rafi et al. 2024). Performance varies substantially by dataset context: laboratory-controlled datasets like CK+ achieve 99-100% accuracy (Kandee, H. M. Abbas, and Hassanein 2021), while real-world datasets like FER-2013 reach only 66% (Arriaga, Valdenegro-Toro, and Plöger 2017), revealing significant generalization challenges across demographics, lighting conditions, and spontaneous expressions (S. M. A. Abbas, Q. Abbas, and Naqi 2025).

Applications span human-computer interaction, healthcare, security and surveillance, driver safety, and IoT systems, with real-time deployment demonstrated on robotic platforms and IoT devices.

Current challenges include limited dataset diversity (Mehrotra, K. P. Singh, and Y. B. Singh 2024), high computational costs limiting deployment (Mehendale 2020), misclassifications between similar emotions (Cirneanu, Popescu, and Iordache 2023), and fundamental questions about the reliability of inferring internal states from facial expressions (Dias et al. 2022).

1.2 Motivation and goals

Our goal is to explore the FER-2013 dataset maximum accuracy limitations, trying to beat the current 66%, and verify whether a properly trained and dedicated CNN could be used for facial emotion recognition in real-time live cams and offline clips video streaming.

Healthcare and medical applications represent a significant domain, with systems designed for monitoring health status in real-time and facilitating psychiatric observations. The medical applications could leverage emotion recognition to analyze emotional responses and improve understanding of patient states.

Security applications are another important field, including access control, authentication, security systems, drunk driver recognition, and surveillance. Driver safety constitutes a specialized security application, with systems specifically designed to identify driver emotions for understanding and managing driver behavior to reduce road accidents.

Another interesting application could be the classification of offline clips based on the percentages of identified facial emotions, For example to classify movies and series not suitable for children or emotionally unstable subjects (too much percentage of detected fear, anger, and the like).

We did not take into consideration neither education nor marketing or workplaces as other application domains, since they could be in stark contrast with current European regulations (AI Act).

2. Data

The FER-2013 dataset (Sambare 2025) consists of 48x48 pixel grayscale images of faces that have been automatically registered to be centered and occupy a similar amount of space in each image. The dataset comprises 35,887 images, consisting of 28,709 examples in the training set and 7,178 examples in the public test set.

2.1 Preprocessing and description

The FER-2013 dataset (Sambare 2025) doesn't need particular preprocessing since all images have been already prepared for Neural Networks' input layer: they are all the same size (48x48 pixels), grayscale, and centered. All images have been already classified into seven emotional categories (Angry, Disgust, Fear, Happy, Sad, Surprise, and Neutral).

The full dataset is available on Kaggle (msambare/fer2013). The total number of images per emotion category, both in the train and the test subsets, are summarized in Table 2.1.

The train set is $\simeq 80\%$ of the total for each emotion category. Notice that "disgust" is the less represented emotion with only 547 total images whereas all other categories count more than four thousand examples. The most represented category is "happy" with almost nine thousand total images. This imbalance (especially between "happy" and "disgust") could be a source of bias and the dataset should be improved, adding more examples for the "disgust" category.

Figure 2.1 shows random examples of images in the FER-2013 train subset along with their the emotional category.

2.2 Potential improvements

Dataset limitations includes reliance on static images limiting applicability (Cirneanu, Popescu, and Iordache 2023), overfitting with large datasets (Mehendale 2020), and lack of diversity preventing bias mitigation toward minority classes (Cirneanu, Popescu, and Iordache 2023).



Figure 2.1: Random examples from FER-2013 train subset with emotional categories.

Table 2.1: Number of images in the FER-2013 dataset.

Emotion	Train	Test	Total	Train %
angry	3,995	958	4,953	80.66
disgust	436	111	547	79.71
fear	4,097	1,024	5,121	80.00
happy	7,215	1,774	8,989	80.26
neutral	4,965	1,233	6,198	80.11
sad	4,830	1,247	6,077	79.48
surprise	3,171	831	4,002	79.24
Total	28,709	7,178	35,887	80.00

The need for large, diverse datasets acquired in dynamic environments remains critical to avoid bias and ensure robustness. Classification rates varying across different databases indicates potential dataset-specific biases, while medical conditions and physical impairments affecting facial expressions introduce additional complexity (Cirneanu, Popescu, and Iordache 2023).

Performance variations across datasets reveal context dependency. Laboratory-controlled datasets like CK+ consistently achieved 99% accuracy (Kandeel, H. M. Abbas, and Hassanein 2021), while the real-world FER-2013 dataset yielded only 66-67%. This 33-percentage-point gap suggests that models trained on posed, well-lit laboratory expressions may not generalize to spontaneous, varied real-world conditions.

3. Methodology

As written before, to achieve our objective we used a Convolutional Neural Network (CNN). A CNN is a deep learning architecture specifically designed for processing grid-like data, such as images, by preserving spatial information. Its architecture is built upon a sequence of specific layers that progressively extract features, starting from simple features (like edges) and combining them into complex features (like whole objects).

3.1 General architecture

3.1.1 Convolutional Layer

This is the most important part in a CNN.

- **Filters (Kernels):** Instead of connecting every neuron to every pixel, this layer uses a set of learnable weights called filters or kernels (e.g., a 3×3 matrix, or a 5×5 matrix).
- **Convolution Operation:** The filter slides (or "convolves") across the input image. At each position, it performs a weighted sum calculation between the weights and the inputs, including a bias (an operation also called "dot product").
- **Feature Map:** The output of previous operation is a feature map (or activation map), which is able to record the locations where specific visual features (like a vertical line or a corner, depending on the kernel) appear in the image.
- **Weight Sharing:** A key property is that the same filter (same weights) is used across the entire image. This allows the network to detect a feature regardless of where it is located (translation invariance).
- **Stride:** This hyperparameter controls how much the filter shifts at each step. A larger stride results in less overlap and smaller output dimensions.

3.1.2 Non-Linearity (Activation Function)

After the convolution, the output is passed through a non-linear activation function. The most common are:

- **ReLU (Rectified Linear Unit):** The standard activation function, more often used. It applies the formula:

$$\text{ReLU}(x) = \max(0, x)$$

The effect is that it keeps positive responses (strong feature matches) and sets negative ones to zero.

- **ELU (Exponential Linear Unit):** An alternative to ReLU that allows negative values. It is defined as:

$$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{if } x \leq 0 \end{cases}$$

where α is a hyperparameter that defines the value to which the ELU saturates for negative inputs. Usually it is set to 1.

3.1.3 Pooling Layer

This layer is used to reduce the spatial dimensions (width and height) of the feature maps, reducing the number of parameters and computation while preserving the most important features in the new feature map.

Max-pooling is the most common technique: a window (e.g., 2×2 or 3×3) slides over the feature map, and only the maximum value within that window is kept.

3.1.4 Flattening

After several rounds of convolution and pooling, the data exists as a compact 3D volume (Height $\times$ Width $\times$ Depth/Channels). The Flatten step converts this 3D volume into a long 1D vector of numbers so it can be fed into a standard neural network.

3.1.5 Fully Connected Layers (Dense Layers)

These are standard neural network layers placed at the end of the CNN. They take the flattened feature vector and combine all the learned features to perform high-level reasoning and classification.

3.1.6 Output Layer (Softmax)

The final layer typically contains one neuron per class (e.g., Angry, Happy, Sad), it applies the Softmax function to convert the raw output scores into probabilities that sum to 1, providing the final classification (e.g., "Angry: 80%").

3.2 Model details

We built our model following a classical VGG-style architecture, an architectural philosophy introduced by the Visual Geometry Group at Oxford University in 2014, characterized by a sequential progression of convolutional blocks that systematically down-sample the spatial dimensions of the input while increasing the depth of the feature maps.

3.2.1 Preprocessing and Augmentation Stage

As we can see from Figure 3.1, the model starts from a preprocessing pipeline embedded directly into the model. The entry point is a Rescaling layer, which normalizes pixel intensity values from the range $[0, 255]$ to $[0, 1]$, facilitating a faster convergence.

Following this normalization, we use a set of Data Augmentation layers:

- RandomFlip: it flips the image horizontally 50% of the time.
- RandomRotation: it randomly rotates the image clockwise or counter-clockwise by up to 14 degrees.
- RandomZoom: this zooms the image in or out by a factor of roughly $\pm 15\%$

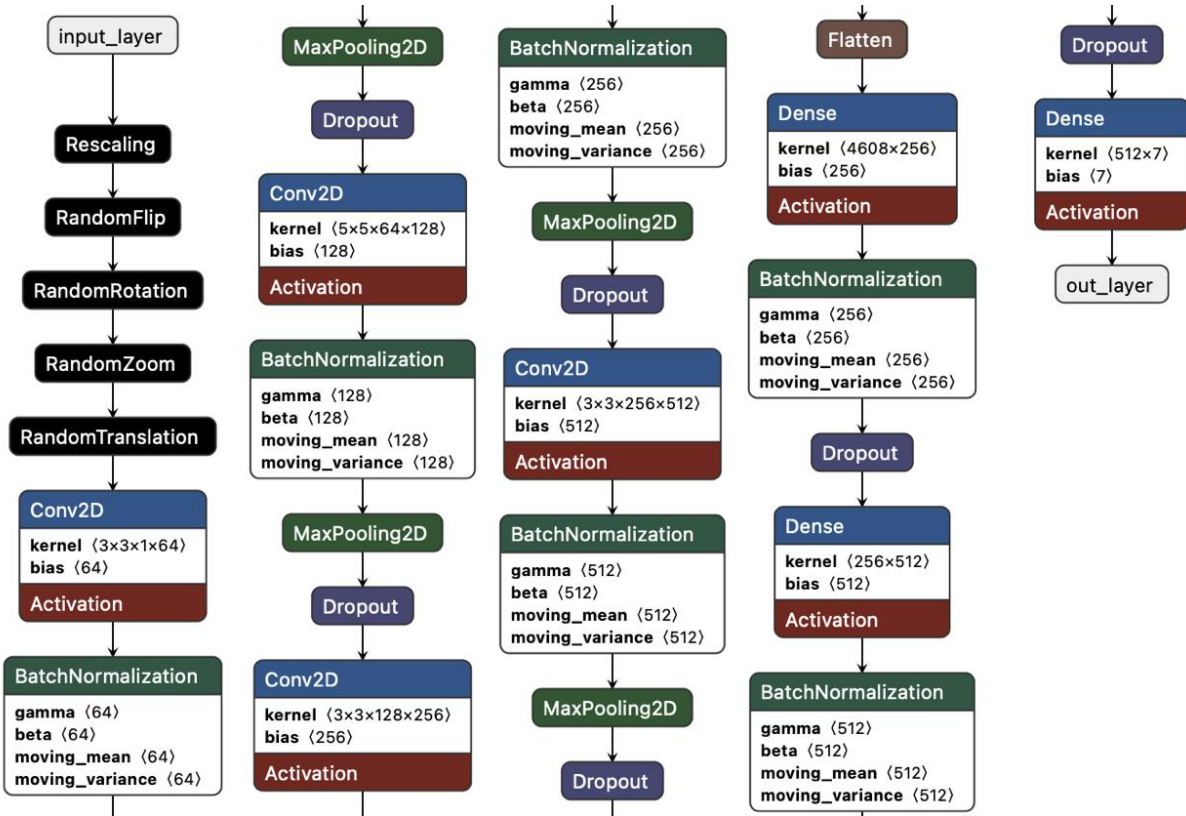


Figure 3.1: Model Architecture

- **RandomTranslation:** it shifts the image up, down, left, or right by up to 15% of the image dimensions.

By doing so we are artificially introducing variance to the training data, forcing the network to learn invariant features rather than memorizing specific pixel arrangements, with the consequence of significantly reducing the risk of overfitting. Specifically, we are training the model also for the cases in which the head is tilted or the face is not fully displayed.

3.2.2 Feature Extraction

The core of the network consists of four distinct convolutional blocks. Each block follows a strict, repetitive design pattern:

Convolution $\rightarrow$ Batch Normalization $\rightarrow$ Max Pooling $\rightarrow$ Dropout

Block 1 and 2 (Low-Level Features): The first block utilizes 64 filters with a standard 3×3 kernel to capture basic edges and textures. The second block doubles the capacity to 128 filters but notably increases the kernel size to 5×5 . This larger receptive field allows the model to capture slightly more complex, medium-scale facial structures earlier.

Block 3 and 4 (High-Level Features): The subsequent blocks continue to double the filter count (256 in Block 3 and 512 in Block 4) reverting to 3×3 kernels. This expansion creates a "pyramid" structure where the data volume becomes deeper (more features) but spatially smaller (due to pooling).

After the convolutions we used ELU (Exponential Linear Unit) activations rather than the traditional ReLU. ELU allows for negative values, which can help pushing mean unit activations closer to zero, speeding up learning. Batch Normalization is applied after every convolution to stabilize the gradient flow. Max Pooling reduces the spatial dimensions by half at each step. Finally, a Dropout rate of 0.25 is applied at the end of every block, randomly disabling 25% of the neurons to ensure the network relies on distributed feature representations and to avoid overfitting.

3.2.3 The Classification Head

Once the feature extraction is complete, the multi-dimensional feature maps are collapsed into a 1D vector via a Flatten layer. This vector is fed into a dense classification head consisting of two fully connected hidden layers, the first containing 256 units and the second expands to 512.

Both dense layers utilize ELU activation, consistently with the convolutional part. They are also regularized using Batch Normalization and Dropout.

The model ends with an **Output Layer** of 7 neurons (matching the 7 emotion classes in FER-2013) activated by a Softmax function. This produces a probability distribution summing to 1.0, allowing the model to make a categorical prediction.

Considering the whole model, the number of parameters is 3,002,887 (corresponding to 11.46 MB).

3.2.4 Training

The model is trained using the Nadam optimizer (Adam with Nesterov momentum), which is generally considered superior for navigating complex loss functions. Adam gives each parameter its own learning rate. If a parameter rarely changes, it gets a big momentum; if it changes constantly, it gets a small movement. The Nesterov momentum part intuition is that, instead of calculating the gradient at the current position, it looks ahead, and calculates the gradient there. This allows the training to correct its course before getting stuck in a local minimum.

The training process also includes a set of callbacks, used to avoid overfitting or uselessly using computational power when the model can't get better.

ModelCheckpoint: Saves only the best version of the model based on validation accuracy.

ReduceLROnPlateau: Dynamically lowers the learning rate if validation accuracy stalls, allowing the model to fine-tune its weights.

EarlyStopping: Stops training if performance stops improving, preventing the model from wasting resources or overfitting in later epochs.

We set the number of epochs to 100. We also set a batch size of 32 since in the FER-2013 dataset there are nearly 29,000 training images and feeding this entire volume of data into the network simultaneously would be computationally infeasible. During the training, the network processes 32 images at a time (the batch), then it calculates the predictions for these 32 examples, computes the average loss, and performs a single update to the model's weights via backpropagation. This approach is known as Mini-Batch Gradient Descent.

In the graphs of Figure 3.2 we can see the increasing of accuracy and the decreasing of the Loss as functions of the Epochs. As we can see from both graphs, in the end the model was overfitting: the Test Accuracy wasn't getting better, so 100 epochs is the correct limit.

3.3 Proof of concept

3.3.1 Confusion Matrix

With the trained model, we used the test set to obtain the confusion matrix of Figure 3.3, along with the recall and precision matrices of figure 3.4.

The overall accuracy is 67%, which, considering the use of black and white pictures, 48*48 pixels, it is a noticeable result.

3.3.2 Test with Images

In Figure 3.5 there's a display of random pictures, correctly labeled from the test set, one for each emotion.

3.3.3 Test with Real Images

The trained model has also been used to predict people's emotions on external pictures, implementing a pre-trained model (CV2) that identifies the face cropping only that area, turning it into black and white, and finally converting it to 48*48 pixels.

The image of Figure 3.6 shows this procedure for 3 pictures.

After this procedure we were able to feed this pictures as inputs to our model. The results were, in the picture's order:

- Happy (confidence: 91.2%)
- Surprise (confidence: 83.22%)
- Fear (confidence: 47.34%)

3.3.4 Working Demo

In the provided files, along with the code used for testing our model with external images, we also included the code for 2 practical applications of this model: The Real Time App and the Video Feed.

In both of them we use CV2 to detect faces, from the webcam input in the first one and from a video in the second one, cropping the images of every single frame and then, after transforming into black and white 48*48 pixels, we feeding them into our model; the result is a real time emotion classification. CV2 works by the means of the Haar cascade classifier (Viola and Jones 2001), a lightweight and efficient machine learning tool that detects the faces using a "cascade approach": if the algorithm finds facial characteristics, it will progressively find more and more features in order to not waste computational power on image regions lacking faces. This allows for fast inference and a good frame rate when actively trying the model with the webcam or video input.

To further improve the video demo, it is crucial to fine tailor the detection parameters, for example to avoid the algorithm finding inexistent faces on walls, clothing and so on. An additional element is the so called histogram normalization: since videos can have an uneven balance of lights and shadows, it sometimes can be helpful to normalize the whole light distribution (called "histogram" in photography) to make the model capture crisp faces with correct lighting on eyes and eyebrows, which are crucial features in face detection and emotion classification.

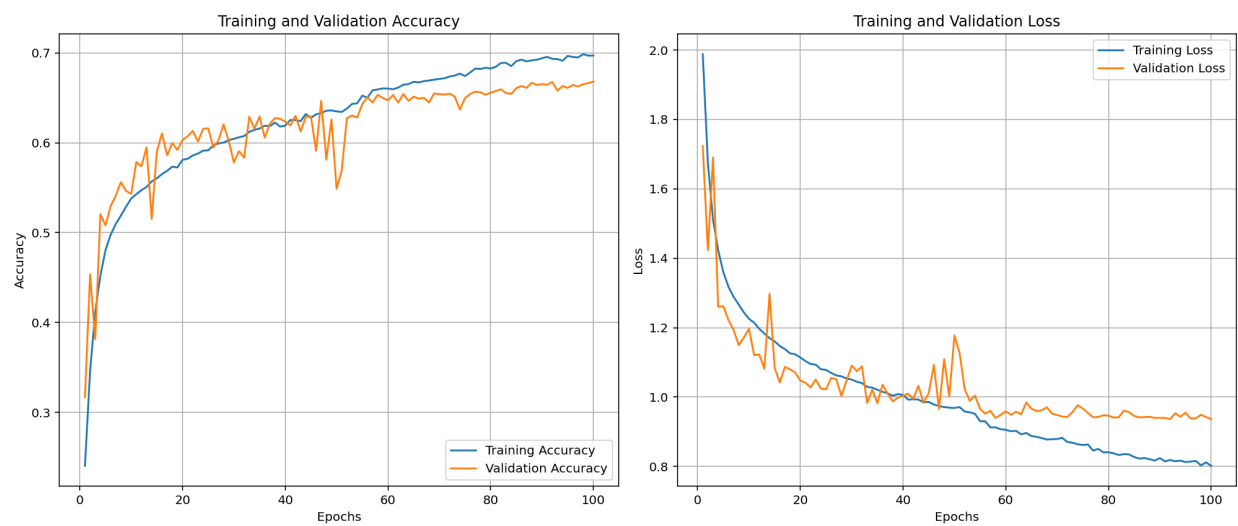


Figure 3.2: Accuracy plots

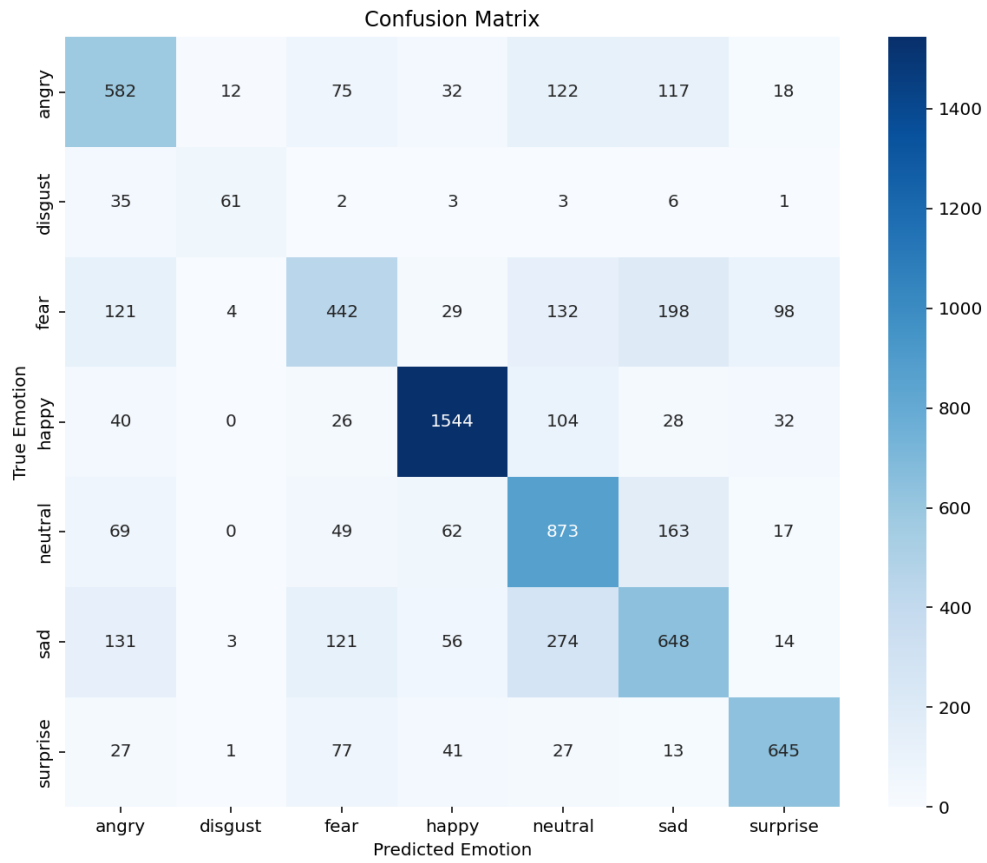


Figure 3.3: Confusion matrix.

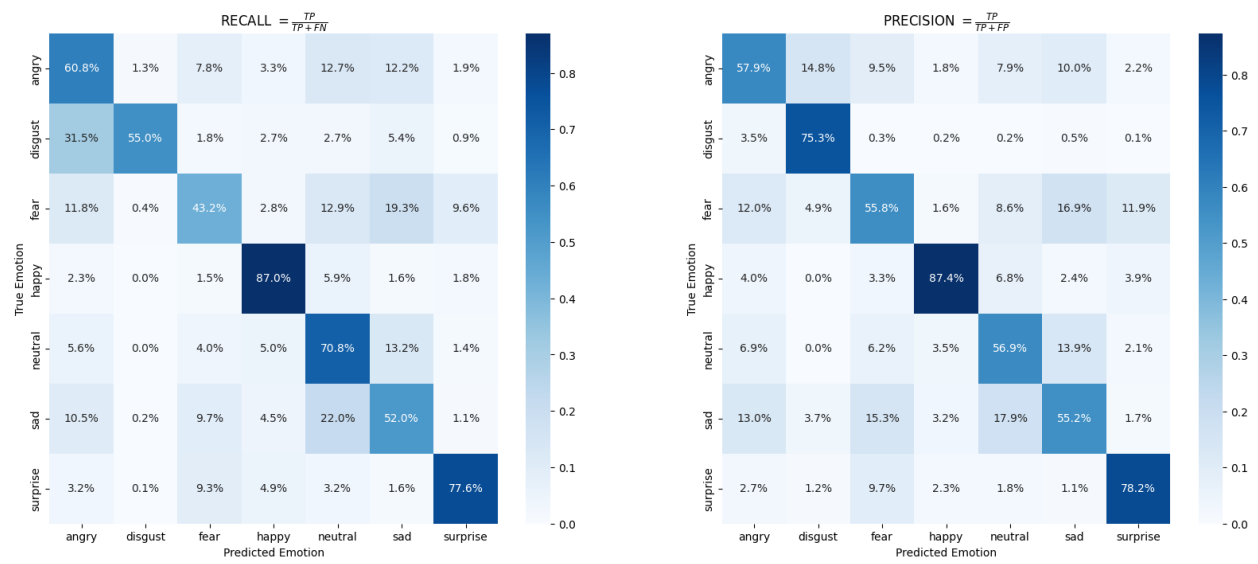
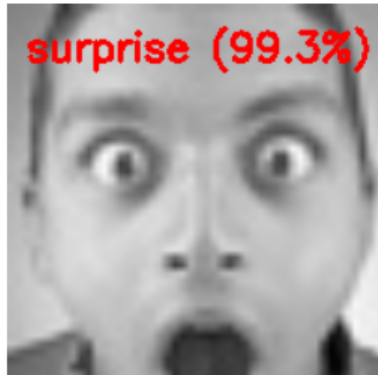


Figure 3.4: Recall and precision matrices.

--- Result ---

Emotion: surprise

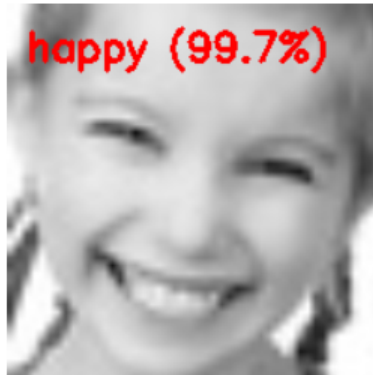
Confidence: 99.32%



--- Result ---

Emotion: happy

Confidence: 99.70%



--- Result ---

Emotion: sad

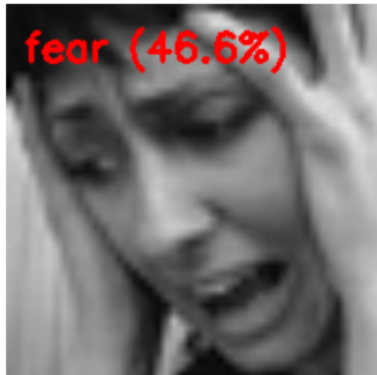
Confidence: 67.60%



--- Result ---

Emotion: fear

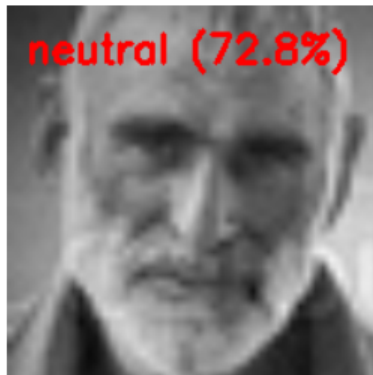
Confidence: 46.63%



--- Result ---

Emotion: neutral

Confidence: 72.83%



--- Result ---

Emotion: angry

Confidence: 96.19%



--- Result ---

Emotion: disgust

Confidence: 98.83%

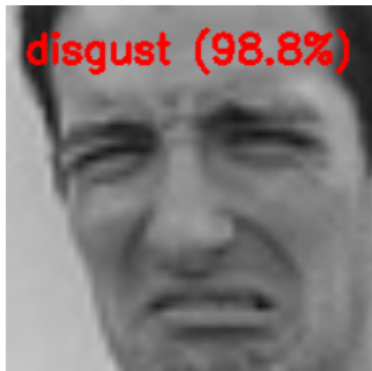


Figure 3.5: Test Images

1. Original (with Crop Box)



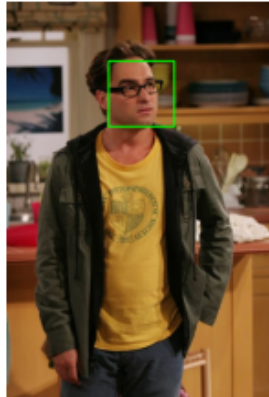
2. Cropped Face (Gray)



3. Final Model Input (48x48)



1. Original (with Crop Box)



2. Cropped Face (Gray)



3. Final Model Input (48x48)



2. Cropped Face (Gray)



3. Final Model Input (48x48)



1. Original (with Crop Box)



Figure 3.6: Examples of CV2 procedure.

4. XAI analysis

Generally speaking, explainability for CNN-based emotion recognition remains underdeveloped, with only few studies implementing dedicated eXplainable Artificial Intelligence (XAI) techniques such as Saliency Maps, Grad-CAM, Class Activation Maps (Kandeel, H. M. Abbas, and Hassanein 2021), or activation visualization (Rafi et al. 2024), while most of the studies lack explicit interpretability methods.

In this study, we opted for a SHapley Additive exPlanation (SHAP) as an attribution method, visualized as pixels' score maps (Lundberg and Lee 2017) that has been identified as one of the most suitable XAI techniques for AI facial emotion recognition (Lorch, Gebele, and Brune 2025).

4.1 Explainability

Given the possible applications, we identified three main stakeholder groups and their *desiderata*, summarized in Table 4.1: "users", i.e. the subjects that will use and apply the AI tool; "affected"; i.e. the subjects that will be the target (so called, "observed") of the applied AI tool; and "regulators", i.e. the institutions that will supervise and regulate the AI tool application.

For all stakeholders, the explanatory information will be of type *contrastive* (highlighting why a facial expression has been categorized as a certain emotion and not another), *counterfactual* (definition of hypothetical cases with different emotion recognition for slightly different facial expressions), and *cautionary* (limits of the AI tool and error risks).

Generally speaking, CNN-based emotion recognition systems are inherently opaque, with a very low transparency level: they lack simulatability, are not decomposable, and exhibit low algorithmic transparency. Thus, the only possible explainability approach is *post-hoc*, involving input-output analyses by extracting representative examples using feature relevance explanations and, eventually, with the advice of an expert human facilitator. For these reasons, SHapley Additive exPlanation (SHAP) has been chosen as the main explanatory tool to analyse selected examples

Table 4.1: Identified stakeholders and their desiderata.

Stakeholder	Desiderata		
Users	Autonomy	Usability	Efficiency
	Trust	Privacy	Security
Affected	Privacy	Security	Consent
	Fairness	Ethics	Efficacy
Regulators	Accountability	Fairness	Security
	Privacy	Ethics	Consent

(Lundberg and Lee 2017) and show which clustered pixels’ areas are more influential in determining the output, i.e. the facial expression categorisation by the trained CNN.

4.2 Regulations and ethics

European regulatory considerations usually are strikingly absent in CNN-based emotion recognition studies: only few studies mention ethical concerns regarding privacy, fairness, and bias (Mehrotra, K. P. Singh, and Y. B. Singh 2024), or discuss privacy and surveillance concerns requiring regulatory oversight (Cirneanu, Popescu, and Iordache 2023), but none specifically address EU AI Act compliance or conformity assessment requirements despite applications in high-risk domains like driver monitoring, access control, and surveillance.

Nevertheless, emotion recognition systems fall under high-risk AI categories in the EU AI Act. The possible applications of such systems (including driver monitoring, security, surveillance, and access control) would likely require conformity assessments, transparency, and human oversight under the Act.

Ethical and privacy concerns emerge as critical considerations. Privacy violations and cultural dependency raise significant issues, with need for regulations to ensure human oversight. Privacy, fairness, and bias in emotion recognition systems require attention. The reliability of

inferring internal emotional states from facial expressions remains an open question, along with uncertainty about the universality of facial expressions for basic emotions (Dias et al. 2022).

4.3 SHAP analysis

SHapley Additive exPlanation (SHAP) is a game-theoretic approach to explaining the output of any machine learning model. It connects local explanations with classical game theory to assign each feature an "importance" value for a particular prediction (Lundberg and Lee 2017). SHAP for images is fundamentally an attribution map, assigning a "score" to each region in the image (usually, clustered pixels) to reflect its importance in determining the trained model's output.

In emotion recognition, SHAP analysis generates an attribution map for each category using an intuitive color palette. Red highlights pixel clusters that strongly support the target emotion, blue indicates those that oppose it, and grey denotes features with minimal influence.

In Figure 4.1, the attribution maps for selected images extracted from the FER2013 test set are shown. Notice that, for all images, clustered pixels associated with the eyes' and mouth shape are the most influential (in red). Notably, across all non-neutral categories, the neutral attribution maps display the strongest concentration of counter-influential clustered pixels in the eye and mouth regions.

These findings align with eye-tracking research regarding the specific regions, often called the "communication triangle" or "T-shape" (eyes and mouth), that humans prioritize when recognizing facial expressions (Yarbus 1967; Schyns, Petro, and Smith 2007; Eisenbarth and Alpers 2011). Nevertheless, while the eyes and mouth are generally considered key areas, studies have demonstrated that these patterns are not universal. For instance, East Asian observers focus almost exclusively on the eyes, relying on subtler cues from the upper face to interpret emotional states (Jack et al. 2009). Consequently, future developments in AI-driven emotion recognition should account for these cultural variations in both the subjects expressing the emotion and the observers interpreting it.

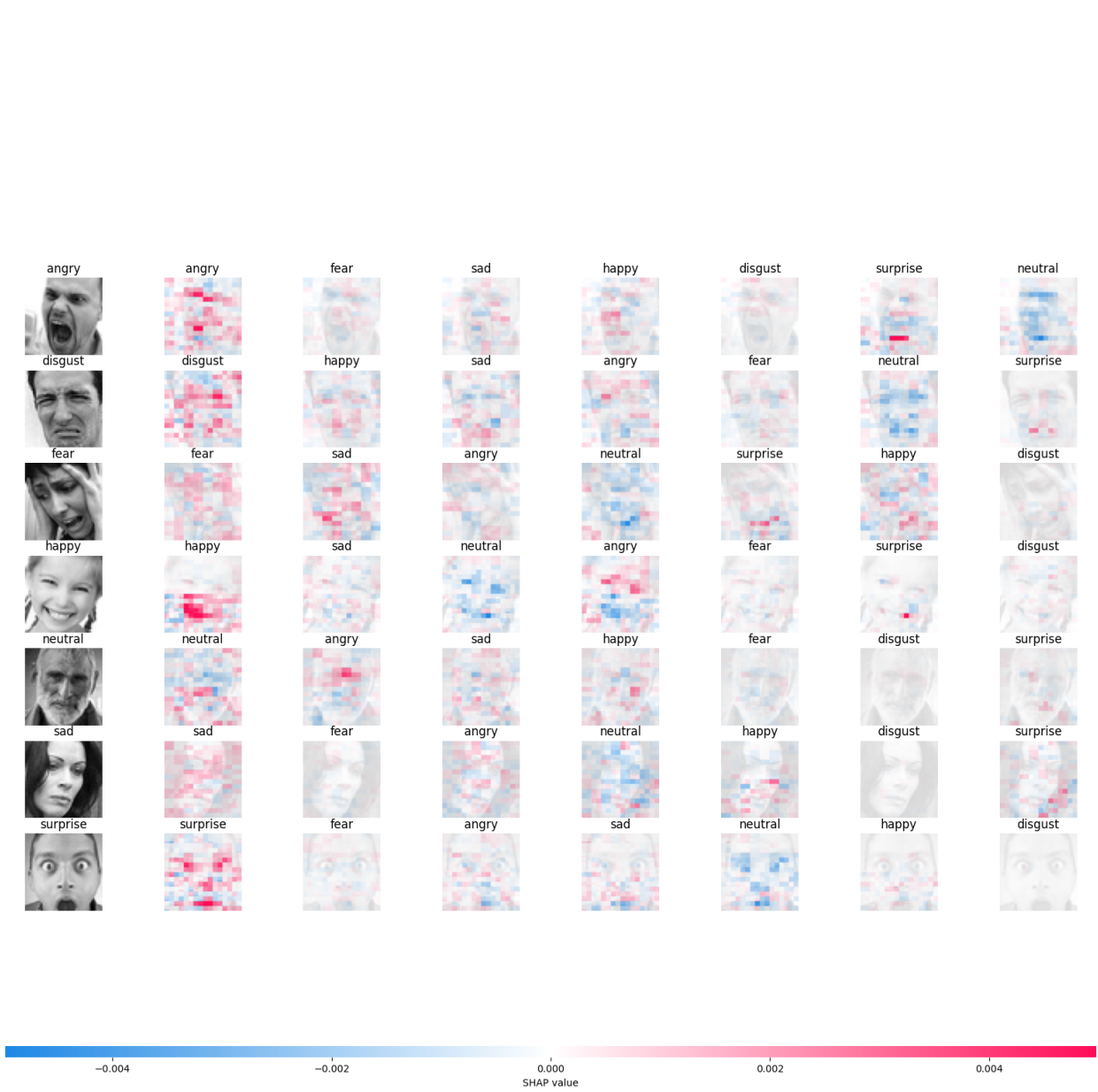


Figure 4.1: SHAP analysis attribution map for examples drawn from the FER2013 test set.

5. Conclusions

In this study, we explored the use of Convolutional Neural Networks (CNNs) for facial emotion recognition using real-world static images, recorded videos, and live camera feeds.

While the results are promising, several factors must be addressed to enhance model performance: increasing image resolution to capture finer details, increasing the training set, reducing the feature imbalance, accounting for the non-universality of facial expressions through additional covariates, and integrating cultural variations in how emotions are expressed and perceived.

Despite the model’s inherent complexity and low explainability, SHAP analysis highlighted the importance of the ”communication triangle” or ”T-shape” (comprising the eyes and mouth) in emotion recognition, aligning with eye-tracking studies revealing that human observers rely on the same facial regions to interpret emotions.

Furthermore, deploying such models in real-world scenarios necessitates a rigorous focus on ethical standards, privacy protections, and regulatory compliance (e.g., EU AI Act).

6. Technical appendix

Computer used for training:

- OS: Fedora Linux 43 (Workstation Edition) x86_64
- CPU: AMD Ryzen 5 3600XT (12) @ 4.63 GHz
- GPU: NVIDIA GeForce RTX 3070 (8 GB of VRAM)

We choose Linux instead of Windows because the latest version of TensorFlow (2.20) does not support the training on the GPU on Windows.

By using the GPU instead of the CPU we reduced the training time of approximately 7 hours into 30 minutes.

To run it we created a virtual environment through Miniconda, in the terminal we run:

```
conda create -n fer_env python=3.10.19
```

We need to use this version of Python because of the compatibility with TensorFlow.

Then we installed the required packages, running these commands:

```
conda activate fer_env
```

```
conda install spyder
```

(We used pip from now on to avoid compatibility issues)

```
pip install tensorflow kagglehub matplotlib
```

Now we are able to run the `train_model.py` script, but it will use only the CPU, to use the GPU we need to install a few more packages:

```
conda install -c conda-forge cudatoolkit=11.8 cudnn=8.9
```

```
python -m pip install "tensorflow[and-cuda]"
```

Then by running the command `spyder` we were able to use the custom env we just created.

To run `realtime_emotion.py` and `VideoRecognition_TBTT.py` we need to install this last package:

```
pip install opencv-python
```

To run the IPython Notebooks, Google Colab is strongly suggested unless you have a fully working environment (e.g. `conda`) with `tensorflow`, `cv2`, and `shap`. Upload to your main Google Drive directory the file *emotion_model_weights.keras*, and the folders *shap*, and *test_img*. You may need to grant Google Colab the authorization to access Google Drive (running block 2 of the ipynb).

List of Figures

2.1	Random examples from FER-2013 train subset with emotional categories.	4
3.1	Model Architecture	9
3.2	Accuracy plots	14
3.3	Confusion matrix.	15
3.4	Recall and precision matrices.	16
3.5	Test Images	17
3.6	Examples of CV2 procedure.	18
4.1	SHAP analysis attribution map for examples drawn from the FER2013 test set. . .	22

Bibliography

- Abbas, Syed Muhammad Aqleem, Qaisar Abbas, and Syed Muhammad Naqi (2025). “EmotionNet-X: An Optimized CNN Architecture for Robust Facial Emotion Analysis”. In: *IEEE Access*.
- Arriaga, Octavio, Matias Valdenegro-Toro, and Paul Plöger (2017). “Real-time convolutional neural networks for emotion and gender classification”. In: *arXiv preprint arXiv:1710.07557*.
- Cirneanu, Andrada-Livia, Dan Popescu, and Dragoş Iordache (2023). “New trends in emotion recognition using image analysis by neural networks, a systematic review”. In: *Sensors* 23.16, p. 7092.
- Dias, William et al. (2022). “Cross-dataset emotion recognition from facial expressions through convolutional neural networks”. In: *Journal of Visual Communication and Image Representation* 82, p. 103395.
- Eisenbarth, Hedwig and Georg W. Alpers (2011). “Happy mouth and sad eyes: Scanning emotional facial expressions”. In: *Emotion* 11.4, pp. 860–865.
- Jack, Rachael E. et al. (2009). “Cultural confusions show that facial expressions are not universal”. In: *Current Biology* 19.18, pp. 1543–1548.
- Kandeel, Amany A, Hazem M Abbas, and Hossam S Hassanein (2021). “Explainable model selection of a convolutional neural network for driver’s facial emotion identification”. In: *International Conference on Pattern Recognition*. Springer, pp. 699–713.
- Lorch, Selina, Jens Gebele, and Philipp Brune (2025). “Towards Trustworthy AI: Evaluating SHAP and LIME for Facial Emotion Recognition”. In.
- Lundberg, Scott M and Su-In Lee (2017). “A unified approach to interpreting model predictions”. In: *Advances in neural information processing systems* 30.
- Mehendale, Ninad (2020). “Facial emotion recognition using convolutional neural networks (FERC)”. In: *SN Applied Sciences* 2.3, p. 446.

- Mehrotra, Manvendra, Kartikeya Pratap Singh, and Youddha Beer Singh (2024). “Facial emotion recognition and detection using convolutional neural networks with low computation cost”. In: *2024 2nd International Conference on Disruptive Technologies (ICDT)*. IEEE, pp. 1349–1354.
- Rafi, M Shahriar Mahmud et al. (2024). “From Image to Emotion: Exploring CNN Architectures for Facial Emotion Recognition”. In: *2024 IEEE International Conference on Computing, Applications and Systems (COMPAS)*. IEEE, pp. 1–6.
- Sambare, Manas (Dec. 2025). *Fer-2013: Learn facial expressions from an image*. URL: <https://www.kaggle.com/datasets/msambare/fer2013>.
- Schyns, Philippe G., Lucy S. Petro, and Marie L. Smith (2007). “Transmission of facial expressions of emotion through space”. In: *Psychological Science* 18.1, pp. 30–35.
- Viola, Paul and Michael Jones (2001). “Rapid object detection using a boosted cascade of simple features”. In: *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*. Vol. 1. IEEE, pp. I–511.
- Yarbus, Alfred L. (1967). *Eye Movements and Vision*. New York: Plenum Press.